Real-Time Sign Language Translator

Kumar Mantha
University of Texas at Arlington
kxm3682@mavs.uta.edu

Nihal Malavalli Lokesh
University of Texas at Arlington
nxl0461@mavs.uta.edu

Abstract

Sign language serves as a primary means of communication for millions of individuals with hearing impairments. To improve communication between sign language users and non signers, we try to create a real-time sign language translation system. The objective is to use computer vision and machine learning techniques to transform hand gestures captured by a camera into text output. The goal is to provide accurate, quick and user friendly translation in real-time circumstances. In a variety of practical applications, this approach could improve inclusion and accessibility for those with hearing and speech impairments.

Keywords: Sign Language Recognition (SLR), Gesture Recognition, Computer Vision (CV), Hand Pose Estimation, Key Point Detection, Landmark Tracking, Natural Language Processing (NLP), Machine Learning (ML), Post training Quantization (PTQ), Quantization Aware Training (QAT).

1. Introduction

People's low usage and comprehension of sign language is an obstacle for millions of deaf and dumb people, even though communication is a basic human necessity. Sign language is a rich, visual language that conveys meaning through body posture, facial expressions and hand movements. Effective communication, however, it can be difficult in commonplace settings like public services, health care or education without an interpreter or a common understanding.

This project focuses on creating a system that can translate sign language gestures into text or speech instantly. We use a camera to capture the movements, the system leverages computer vision and machine learning to recognize and understand the signs quickly, making communication smoother and more accessible.

Our approach tackles the communication gap by using a camera to "see" the signs someone makes with their hands. The system then analyzes these movements in real time, by applying CV techniques to detect key hand landmarks. With help of ML models, recognize the gesture and convert it to text. This provides an accessible solution, leveraging techniques like hand pose estimation and gesture recognition to support better communication.

2. Related Work

Translating sign language into readable or spoken language has become an active area of research in recent years, driven largely by progress in computer vision and deep learning. Early systems often relied on traditional image processing methods such as color segmentation or contour matching to detect hand shapes. While these are effective in controlled settings, these techniques may struggle with real-world challenges like change in lighting, complex backgrounds or variations in hand orientation [1].

The adoption of deep learning marked a turning point for gesture recognition. Convolutional Neural Networks (CNNs) have shown strong performance in recognizing static hand signs such as alphabets and digits. For example, Pigou et al. [2] demonstrated that CNNs could significantly out-perform handcrafted features by leveraging depth and RGB data.

As research moved beyond static gestures, temporal models like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks became mainstream for handling dynamic gestures and full sentence level recognition. Molchanov et al. [3] combined 3D CNNs with LSTMs to capture spatiotemporal information in video sequences, establishing an early framework for continuous sign language recognition.

More recently, lightweight tools such as MediaPipe Hands [4] have enabled robust, real-time hand tracking on everyday devices. By providing 21 landmark points per frame, MediaPipe offers a fast and reliable way to capture hand poses under varied conditions, making it especially attractive for real-time applications.

Despite these advances, existing systems often face trade-offs between accuracy, efficiency and scalability. Many

systems are still computationally heavy, optimized only for limited sign sensor impractical for real-time use. Our work builds on these foundations by designing a lightweight framework that aims to recognize both static and eventually dynamic signs in real time, making sign language recognition more accessible and practical for everyday use.

To further improve efficiency and enable deployment on resource-constrained devices, recent research has explored neural network quantization techniques. Quantization reduces the precision of network weights and activations, allowing models to run faster and with lower memory requirements while maintaining competitive accuracy [5]. Two commonly used approaches are Post Training Quantization (PTQ) and Quantization Aware Training (QAT) each offering distinct trade-offs between model size, latency, and performance [6]. Integrating such quantization strategies into sign language recognition pipelines has the potential to create lightweight, real-time models that can operate on edge devices without sacrificing recognition accuracy aligning with the goals of making sign language translation more practical for everyday use.

3. Problem Statement

The objective of this project is to develop a real-time system capable of translating **sign language gestures** into corresponding **textual output**, using visual input from a standard camera. The system must accurately recognize individual signs primarily static hand gestures such as letters from the American Sign Language (ASL) alphabet with the potential for future extension to dynamic gestures and sentence level translation.

Formally, given a sequence of video frames captured from a live camera feed, the system should: (I) Detect and track the hand(s), (ii) Extract relevant spatial features (e.g., hand landmarks or gesture descriptors), (iii) Classify the gesture into one of the predefined sign classes and output the corresponding English character, word or phrase. The problem must be solved under constraints of **low latency**, **high accuracy** and **robustness in real world conditions**, such as variation in lighting, background, hand size or orientation.

3.1. Datasets

To support both training and evaluation, we rely on two widely used sign language datasets. The first is the **ASL Alphabet Dataset** [1], which contains more than 87,000 labeled images covering all 26 letters of the ASL alphabet. The dataset was collected from different individuals in varied environments, making it valuable for building models that can generalize to real world conditions. The second is the **Sign Language MNIST dataset** [2], a simplified benchmark consisting of 28×28 grayscale images of ASL signs A–Y (excluding J and Z). While smaller in scope, this dataset is particularly useful for establishing baseline experiments and testing lightweight models before scaling more complex data.

Each class directory contains several thousand samples and the distribution across classes is approximately uniform. The training set is used to develop both the baseline and quantized CNN models, while the test set provides the benchmark for accuracy, latency and model efficiency comparisons.

Figure 1: Dataset Structure.

All images are in **RGB format** with a typical resolution of **200 × 200 pixels**, though dimensions may vary slightly across samples. The hands are photographed in diverse lighting environments, resulting in a mixture of shadowed, bright and neutral tones. Backgrounds range from plain walls to textured surfaces, introducing realistic noise that makes the dataset suitable for real-world scenarios.

Figure 2: Example Images for letter ‘A’ from ASL dataset

3.2. Methodology

We will start with a lightweight CNN backbone such as **MobileNetV3-Large**, **MobileNetV3-Small**, **Efficient Net-Lite** or a **custom compact CNN** trained on RGB images and MediaPipe hand landmarks. The primary objective is to design a computationally efficient yet accurate model for real-time sign language recognition on

edge devices. To ensure a fair performance evaluation, we establish two baseline models: (A) a MediaPipe hand landmark based approach using classical classifiers such as **SVM** or **Random Forest** for alphabet classification or (B) A **float32 CNN classifier** operating directly on RGB image inputs.

To improve the model's efficiency for real-time deployment, we employ **quantization techniques** that reduce the numerical precision of weights and activations from 32-bit floating point to 8-bit integer representations. We explore two strategies: **Post Training Quantization (PTQ)** and **Quantization Aware Training (QAT)**. In PTQ, quantization is applied after the model is trained, using a small calibration dataset to determine the appropriate scaling parameters. In contrast, QAT simulates quantization effects during training, allowing the network to adapt to lower precision and recover most of the accuracy lost through PTQ. Both approaches yield a compact, low-latency model suitable for on-device inference.

Our training pipeline begins with preprocessing each frame using **MediaPipe Hands** to extract and crop the hand region, reducing background noise and improving data consistency. For the RGB based CNN, the cropped images are resized to 128×128 and normalized to $[0,1]$. For the landmark based models, 21 key points per hand (x, y, z) are extracted and normalized with respect to the hand bounding box. To improve generalization, we apply standard data augmentations such as random rotation, brightness variation, or horizontal flips.

During our model, we first train the baseline float32 model to convergence and evaluate its accuracy. The model is then converted using PTQ to assess performance degradation. If significant accuracy loss is observed, we perform QAT fine tuning using simulated quantization operations integrated into the training graph. We monitor both validation accuracy and model efficiency metrics, including model size, inference latency, or frames per second (FPS) on the target edge device.

Finally, the quantized models are compared against the float32 baseline and the classical landmark based classifiers. We evaluate each model's **accuracy, model size, latency or energy efficiency**, providing a detailed trade-off analysis. The best performing lightweight quantized CNN is then selected for deployment in the real-time sign language translation system.

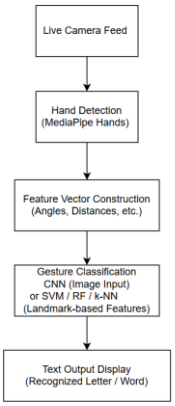

Figure 3: Generalized working of a gesture detection model.

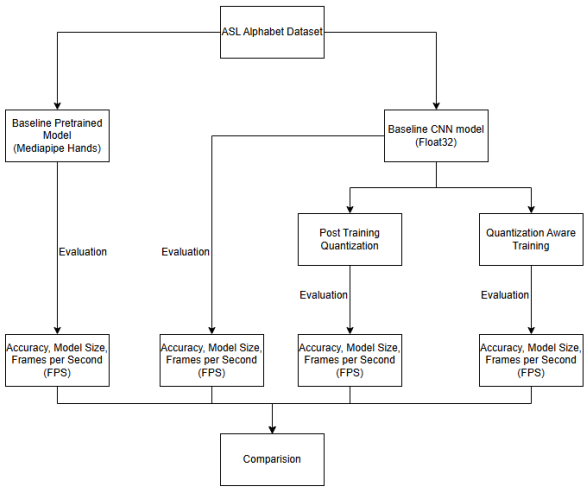

Figure 4: Methodology Flowchart

3.3. Model Architecture

Our proposed model architecture is designed to achieve high accuracy while maintaining a compact footprint suitable for mobile deployment. We experiment with both pretrained lightweight architecture and a custom designed compact CNN to assess the trade-offs between model complexity and performance.

To evaluate the effectiveness of quantization for lightweight vision models, we develop and compare four model variants designed around varying levels of complexity and precision.

Baseline 1 – MediaPipe + CNN Classifier:

We use MediaPipe Hands as a pretrained feature extractor to detect and generate hand landmarks. These landmark coordinates are then passed to traditional classifiers (CNN)

for alphabet classification. This configuration serves as our non-deep learning baseline, representing a low computation reference point.

The implementation extends the baseline CNN by integrating MediaPipe based preprocessing and a flexible training framework optimized for GPU and mixed precision environments. The dataset is dynamically loaded with augmentation, resized to 64×64, and split into training, validation and test sets. The architecture consists of four convolutional blocks (32–256 filters), Batch Normalization layers for stable convergence, and a dense classification head with dropout regularization. The script includes automatic GPU configuration, mixed precision support for acceleration, early stopping, learning rate scheduling, and model checkpointing. Additionally, a hyperparameter search module tests variations of filters, dropout rates, dense units and learning rates to select the best performing configuration. This model serves as a more optimized and scalable alternative to the baseline float32 CNN, improving training speed and adaptability for deployment.

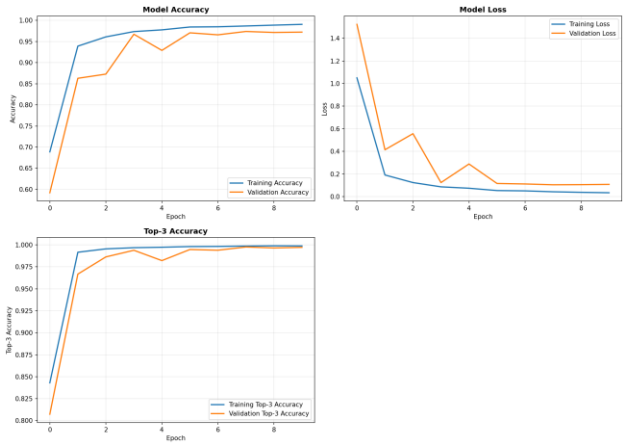

Figure 5: Plots for Mediapipe-Hands + CNN model

Baseline 2 – Standard CNN (Float32):

A general convolutional neural network trained on RGB images is used to establish the baseline for quantization experiments. This model operates in full 32-bit floating point precision and represents the upper bound of accuracy before compression.

The baseline classification model was implemented as a sequential CNN trained on 200×200 RGB ASL images using an 80:20 train–validation split. The network consists of four convolutional blocks with increasing filter sizes (32, 64, 128, 256), each followed by MaxPooling and Batch Normalization to improve convergence stability and reduce overfitting. The extracted spatial features are flattened and passed through a 256-unit dense layer with dropout (0.5) before classification via a softmax output layer across all gesture classes. The model was trained for 8 epochs using Adam optimizer (lr = 1e-4) with categorical cross-entropy loss, and augmented input data to improve robustness. This float32 network serves as the baseline for later PTQ and QAT quantization experiments.

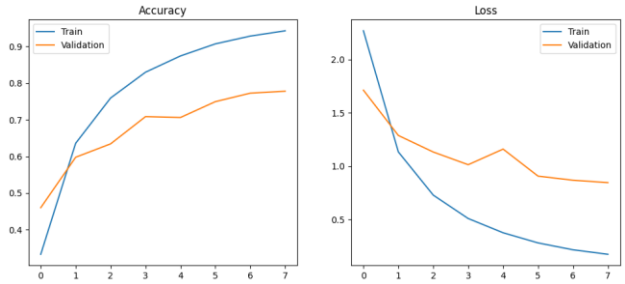

Figure 6: Plots for Baseline CNN model (FP32)

Model 3 – PTQ CNN (Post Training Quantization):

The float32 CNN is converted to an 8-bit integer model using Post Training Quantization (PTQ). This approach compresses the trained network without retraining, using a small calibration dataset to determine optimal quantization parameters. It provides a fast, low overhead path to model size and latency reduction.

To compress the baseline float32 CNN and enable deployment on resource constrained devices, Post Training Quantization (PTQ) was applied using TensorFlow Lite. The trained model was first converted to FP16 format to reduce memory footprint while preserving floating point computation. Further optimization was performed using full INT8 quantization, utilizing a representative calibration dataset sampled from validation batches. The script evaluates all three variants Float32, FP16 TFLite, and INT8 TFLite reporting accuracy, precision, recall, and F1-score on the validation set. PTQ successfully reduced model size significantly (Float32 → FP16/INT8) with minimal impact on accuracy, demonstrating its effectiveness for lightweight ASL gesture inference and real-time deployment.

Model 4 – QAT CNN (Quantization Aware Training): To recover potential accuracy loss from PTQ, we train a Quantization Aware CNN, which simulates quantization effects during training. This model learns to adapt to

reduced numerical precision, balancing accuracy with deployment efficiency.

To further minimize performance loss from quantization, Quantization Aware Training (QAT) was implemented by injecting fake_quant layers after convolutional and dense blocks, allowing the model to simulate INT8 behavior during training. The CNN architecture mirrors the baseline configuration (32→256 filters, BatchNorm, Dropout) but trains under quantization constraints so the network learns weights robust to reduced precision. The model was trained for 8 epochs using the same 80:20 split and augmentation pipeline as the baseline, then exported as both float32 and fully quantized INT8 TFLite versions using a representative dataset for calibration. QAT improves post quantization stability, typically retaining accuracy closer to the original FP32 model while achieving a significantly smaller footprint, making it more deployment friendly than PTQ for real-time gesture recognition.

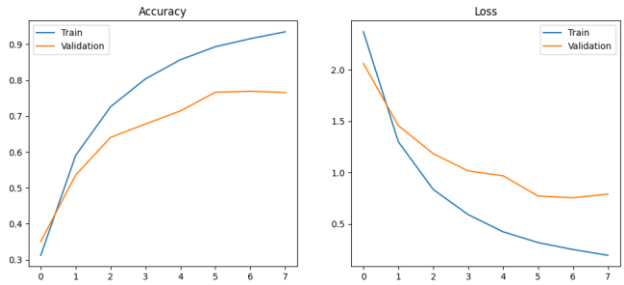

Figure 7: Plots for QAT model (INT8)

Across all configurations, the focus is on achieving a strong trade-off between model accuracy, size and inference latency for real time sign language recognition on resource constrained devices.

3.4. Image Generalization

As shown in Figure 2 images can vary with respect to many aspects when we consider hand gestures. The models were trained using RGB images under diverse backgrounds and lighting conditions. Data augmentation involves creating synthetic variations of your existing training images to expand the dataset's size and diversity. This tricks the model into thinking it's seeing more unique examples, forcing it to learn the true, underlying features of the objects instead of memorizing specific pixels or orientations. This restricts the model from overfitting. We added a small code which randomly changes factors of images such as scale, rotation and tilted hand orientation, brightness etc. so that no two images given to the model were same. This helps to enhance the model's robustness by ensuring the model learns features that are invariant to minor, realistic changes.. The model correctly classified a majority of samples, showing that Data augmentation + CNN features provided strong generalization capability. However, the performance dropped when background clutter was high, hands were partially occluded, or camera distance was varied significantly.

3.5. Evaluation Metrics

To assess model performance comprehensively, we evaluate each configuration using a combination of accuracy based and efficiency based metrics. The primary metric is classification accuracy, computed on the test set to measure the model's ability to correctly recognize hand signs. To further analyze robustness, we examine per class precision, recall and F1-score, which highlight the performance consistency across all alphabet classes. In addition to predictive performance, we evaluate model size (in MB) and frames per second (FPS) on the target edge device to quantify computational efficiency. The accuracy efficiency trade-off between the float32, PTQ and QAT models serves as a key indicator of quantization effectiveness. All metrics are averaged over multiple runs to ensure reliability and reproducibility of results.

Model	Accuracy	Precision	Recall	F1 Score	Size in MB
Mediapipe + CNN	0.9310	0.9326	0.9310	0.9327	67.61
CNN Baseline	0.7779	0.8061	0.7779	0.7772	118.11
PTQ Model	0.7572	0.7938	0.7572	0.7595	9.86
QAT Model	0.7820	0.8070	0.7820	0.7829	9.86

Table 1: Evaluation Metrics for all models.

3.6. Results and Performance Analysis

It is important to note that the Mediapipe based model was trained for 15 epochs with a smaller input size of 64×64, whereas the baseline CNN, PTQ, and QAT models were trained only for 8 epochs using larger 200×200 images. This difference in training duration and input resolution directly impacts convergence and feature representation quality, explaining part of the performance gap observed in the results. The Mediapipe model benefits from compact landmark features, enabling faster learning and requiring less computational effort to achieve high accuracy (93.10%). In contrast, the baseline CNN learns directly from raw images, requiring more epochs to extract robust spatial features; with only eight training epochs, it reached 77.79% but is expected to improve significantly with longer training. After quantization, PTQ reduced model size from 118 MB to 9.86 MB, with a small accuracy drop to 75.72%, while QAT preserved more accuracy (78.20%) at the same model size by simulating quantization noise

during training. These results confirm that QAT is more reliable than PTQ for maintaining performance post compression, and both quantized versions are better suited for real-time deployment scenarios due to their lightweight footprint.

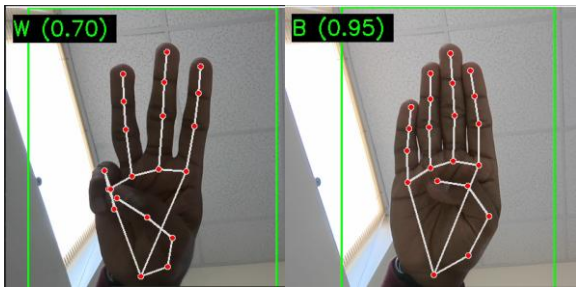

Figure 8: Output of CNN+Mediapipe model

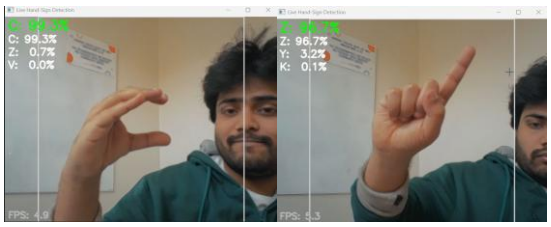

Figure 9: Output of QAT model

3.7. Conclusion

This project successfully developed a real-time ASL alphabet recognition system using CNN models with quantization optimization. The baseline CNN achieved a descent accuracy and QAT enabled compression without major performance loss. Our final model is lightweight, fast and deployable for real-time applications. Although data augmentation and MediaPipe preprocessing improve robustness, accuracy still degrades when background clutter is high, hands are partially occluded or the camera hand distance changes significantly.

3.8. Future Scope

We can extend beyond ASL alphabet to full ASL vocabulary and later to other sign languages (BSL, ISL etc). Application of domain adaptation techniques so the model generalizes better across different cameras, backgrounds and skin tones without retraining from scratch. Conduct user studies with Deaf and Hard-of-Hearing communities to evaluate usefulness, cultural appropriateness and real world gaps. Use feedback to refine gesture sets, UX and error modes.

Contribution

The first phase of the Sign Language translation project involved a focused division of labor to establish the foundational environment and explore initial methodologies. Nihal established the project's foundational infrastructure, including setting up the training pipeline and implementing the data handling procedures necessary for the gesture dataset. He was primarily responsible for selecting, implementing and establishing the baseline hand gesture detection model and conducting its initial performance evaluation to set the project's benchmark and inform architectural direction.

Kumar led the architectural exploration focused on model lightweighting for deployment efficiency. This involved literature review and research into model optimization techniques, specifically conducting an in depth exploration of quantization concepts and their feasibility for integrating into the hand gesture detection framework. He formulated testing methodology for these lightweighting methods.

References

[1] T. Starner, J. Weaver and A. Pentland, "Real-time American sign language recognition using desk and wearable computer based video," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 20, no. 12, pp. 1371-1375, Dec. 1998, doi: 10.1109/34.735811.
keywords: {Handicapped aids;Wearable computers;Hidden Markov models;Cameras;Speech recognition;Face recognition;Computer vision;Computer Society;Real time systems;Pattern recognition},

[2] Pigou, L., Dieleman, S., Kindermans, PJ., Schrauwen, B. (2015). Sign Language Recognition Using Convolutional Neural Networks. In: Agapito, L., Bronstein, M., Rother, C. (eds) Computer Vision - ECCV 2014 Workshops. ECCV 2014. Lecture Notes in Computer Science(), vol 8925. Springer, Cham. https://doi.org/10.1007/978-3-319-16178-5_40

[3] Molchanov, Pavlo & Gupta, Shalini & Kim, Kihwan & Kautz, Jan. (2015). Hand gesture recognition with 3D convolutional neural networks. 1-7. 10.1109/CVPRW.2015.7301342.

[4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Changor M. Grundmann, *MediaPipe Hands: On-device Real-time Hand Tracking*, arXiv preprint arXiv:2006.10214, 2020.

[5] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos and T. Blankevoort, "A White Paper on Neural Network Quantization," *arXiv preprint arXiv:2106.08295*, 2021.

[6] J. Hasan, "Optimizing Large Language Models through Quantization: A Comparative Analysis of PTQ and QAT Techniques," *arXiv preprint arXiv:2411.06084*, 2024. [Online]. Available: https://arxiv.org/pdf/2411.06084